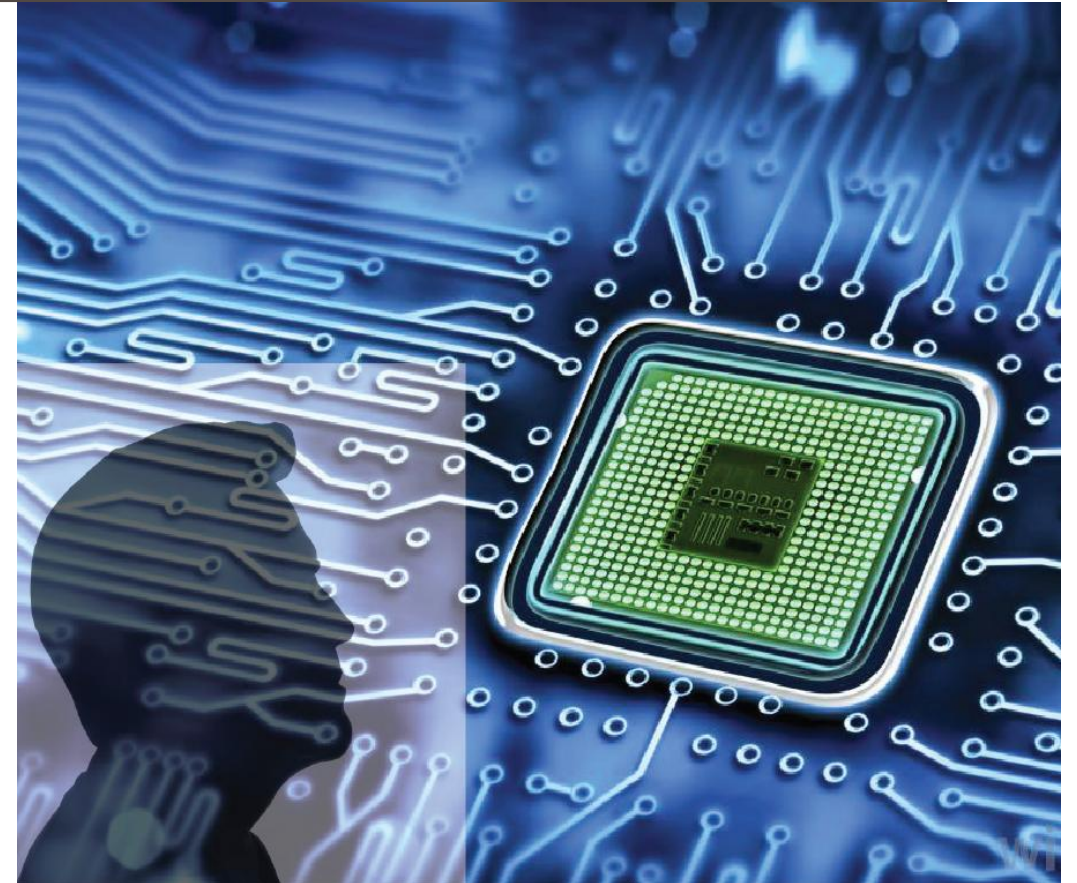


ADVANCED DIGITAL DESIGN AND VERIFICATION

Week-9 Lecture-2 SystemVerilog

Sneh Saurabh
16th October, 2025



Clarification

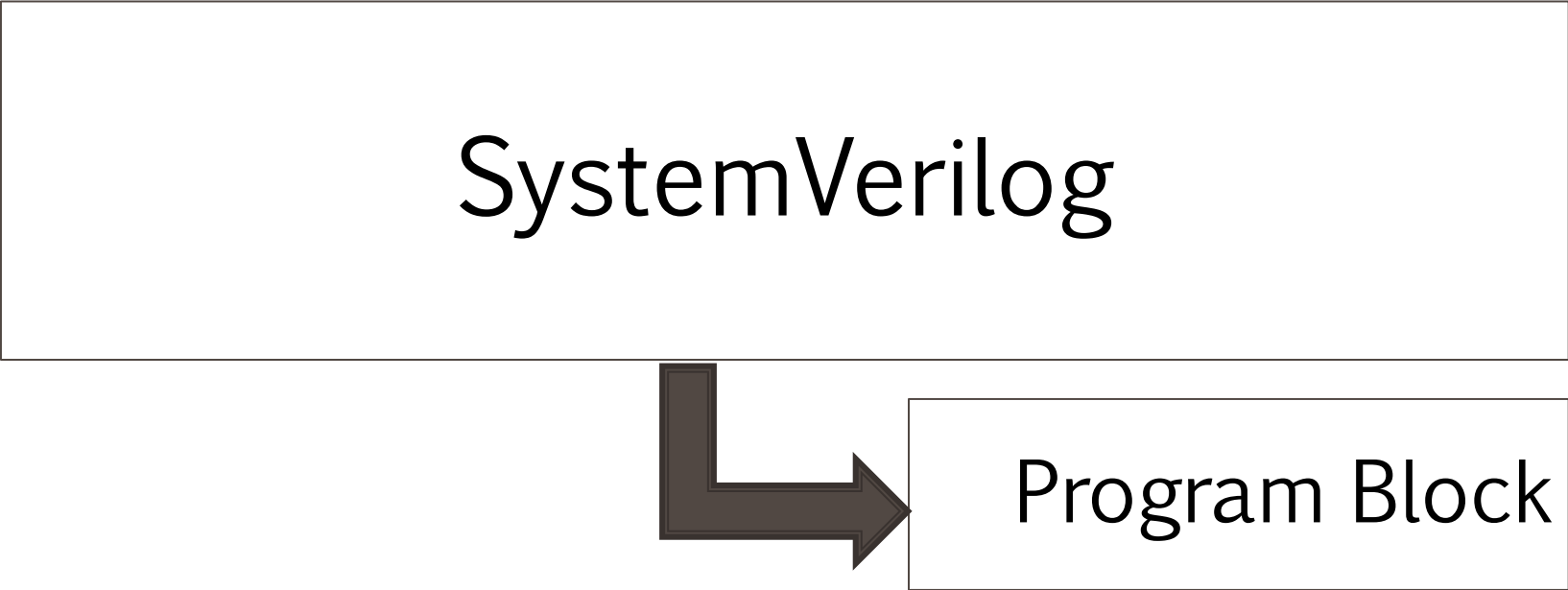
```
1 class Base;
2   virtual function void display();
3     $display("In Base class");
4   endfunction
5 endclass
```

```
|ncsim> run
|In Derived class
```

```
7 class Derived extends Base;
8   virtual function void display();
9     $display("In Derived class");
10  endfunction
11  function void nonVirtualDisplay();
12    $display("In Derived class: nonVirtualDisplay");
13  endfunction
14 endclass
```

```
module top;
  Base b;
  Derived d;
  initial begin
    d = new;
    b = d;
    b.display();
    // b.nonVirtualDisplay();
  end
endmodule
```

```
file: base_call.sv
      b.nonVirtualDisplay();
      |
ncvlog: *E,NOTCLM (base_call.sv,25|19): nonVirtualDisplay is not a class item.
```



```
graph TD; A[SystemVerilog] --> B[Program Block]
```

SystemVerilog

Program Block

SystemVerilog: Timing Regions

SystemVerilog introduces new timing regions in the Stratified Event Queue

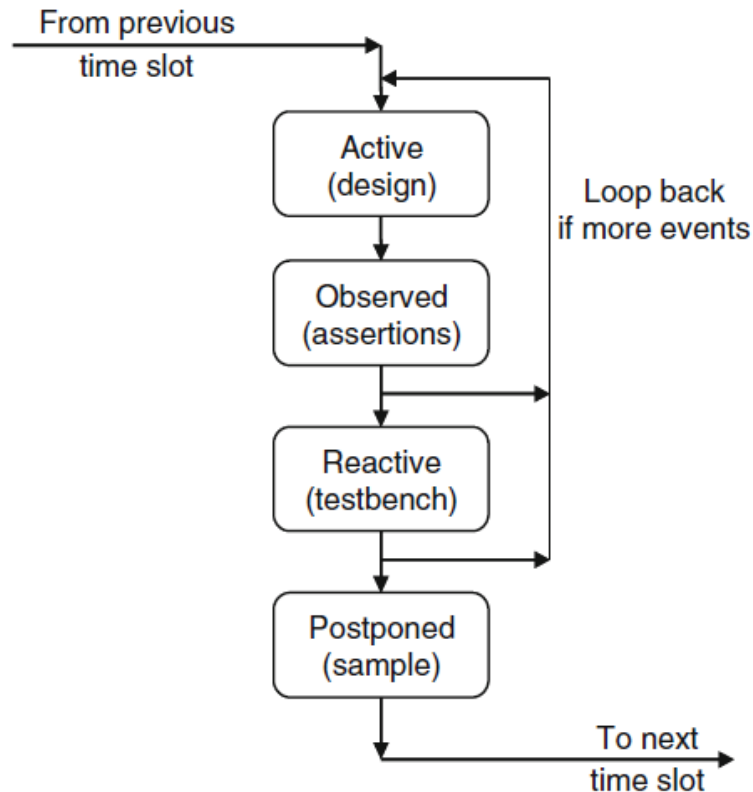
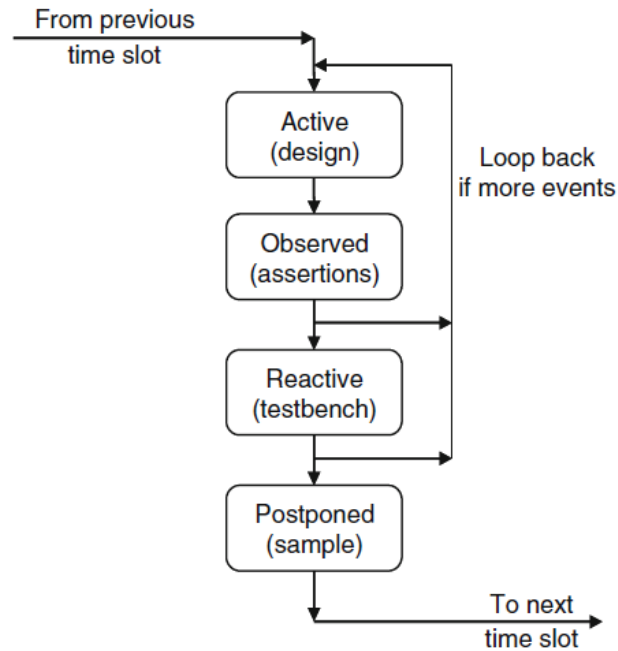


Table 4.1 Primary SystemVerilog scheduling regions

<i>Name</i>	<i>Activity</i>
Active	Simulation of design code in modules
Observed	Evaluation of SystemVerilog Assertions
Reactive	Execution of testbench code in programs
Postponed	Sampling design signals for testbench input

SystemVerilog: Timing Regions



```

1 program automatic test(output bit toggle);
2 initial begin
3     $display("In reactive region toggle=%0d time:%0d", toggle, $time);
4     toggle = 1;
5     #1;
6 end
7 endprogram

9 module dut (input bit toggle);
10 initial begin
11     $display("In active region toggle=%0d time:%0d", toggle, $time);
12 end
13 always @(posedge toggle) begin
14     $display("Re-enetring active region region toggle=%0d time:%0d", toggle, $time);
15 end
16 endmodule
    
```

```

19 module top;
20     wire toggle;
21     dut d1(toggle);
22     test t1(toggle);
23 endmodule
    
```

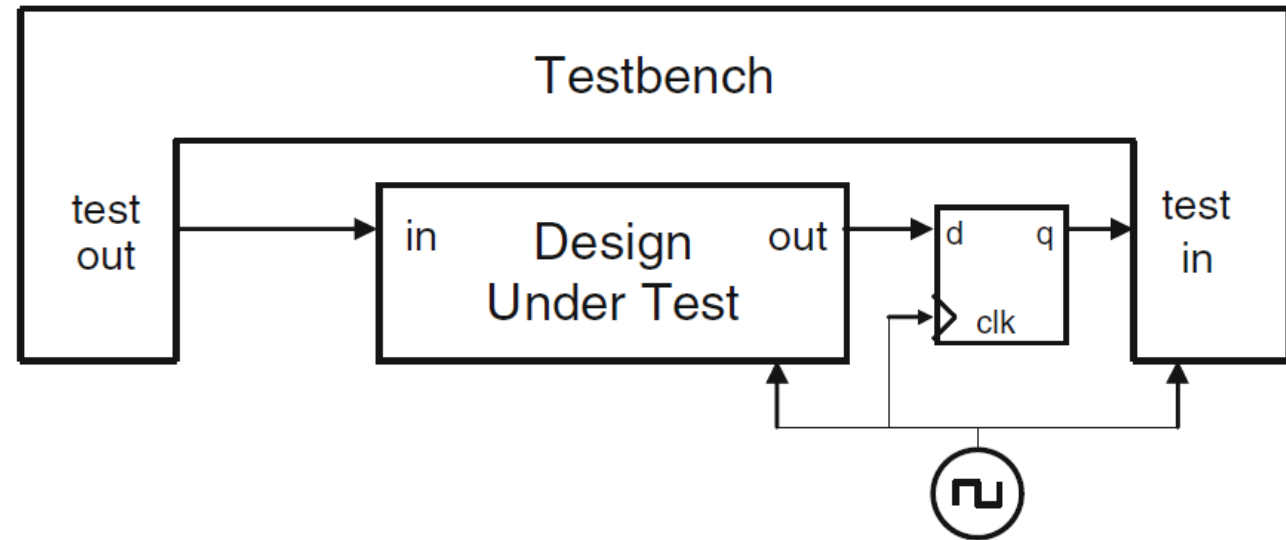
```

In active region toggle=0 time:0
In reactive region toggle=0 time:0
Re-enetring active region region toggle=1 time:0
    
```

Clocking Block and Program Block

- Clocking block drives outputs with a delay of #0
 - Generates stimulus in the Reactive Region
 - Signal goes directly to DUT, and is evaluated in the Active region of the same time slot

- Clocking block samples input with a skew of #1step
 - Signal sampled in the postponed region of the previous time slot (before any design activity)
 - Reads values before the clock changes



- In effect, SystemVerilog inserts a flip-flop between the testbench and DUT
- Using Clocking Block and Program Block, we can easily avoid the race conditions

Sampling Signal in Testbench

```
interface arb_if(input bit clk);
    logic [1:0] grant, request;
    logic reset;

    clocking cb @(posedge clk); // Declare cb
        output request;
        input grant;
    endclocking

    modport TEST (clocking cb, // Use cb
        output reset);

    modport DUT (input request, reset, clk, output grant)

endinterface
```

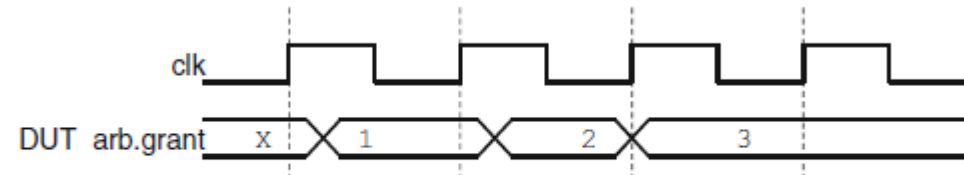
```
program automatic test(arb_if.TEST arbif);
initial begin
    @arbif.cb;
    $display("@%0t: grant=%h", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: grant=%h", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: grant=%h", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: grant=%h", $time, arbif.cb.grant);
end
endprogram : test
```

```
module arb_mod (arb_if.DUT arbif);
    initial
        fork
            # 70 arbif.grant = 1;
            # 170 arbif.grant = 2;
            # 250 arbif.grant = 3;
        join
    endmodule
```

```
module top;
    bit clk;
    always #50 clk = ~clk;

    arb_if arbif(clk);

    arb_mod a1 (arbif.DUT);
    test t1(arbif.TEST);
endmodule
```



- Samples values just before the clock edges

```
@50: grant=x
@150: grant=1
@250: grant=2
@350: grant=3
```

Driving Signal from the Testbench

```
interface arb_if(input bit clk);
    logic [1:0] grant, request;
    logic reset;

    clocking cb @(posedge clk); // Declare cb
        output request;
        input grant;
    endclocking

    modport TEST (clocking cb, // Use cb
        output reset);

    modport DUT (input request, reset, clk, output grant);
endinterface

program automatic test(arb_if.TEST arbif);
    initial fork
        # 70 arbif.cb.request <= 3;
        # 170 arbif.cb.request <= 2;
        # 250 arbif.cb.request <= 1;
        # 500 $finish;
    join
endprogram : test
```

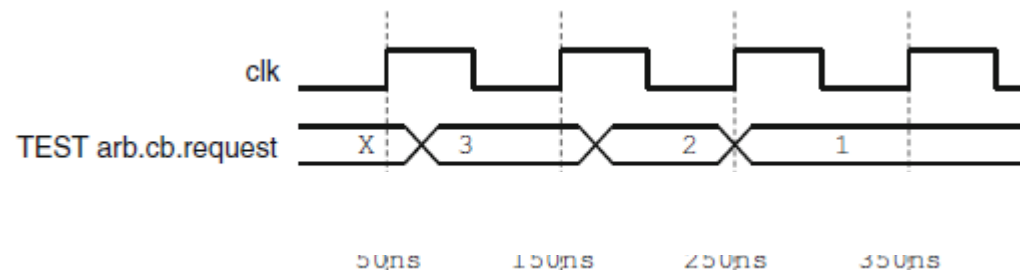
```
module arb_mod (arb_if.DUT arbif);
    initial
        $monitor("@%0t: req=%h", $time, arbif.request);
endmodule
```

```
module top;
    bit clk;
    always #50 clk = ~clk;

    arb_if arbif(clk);

    arb_mod a1 (arbif.DUT);
    test t1(arbif.TEST);
endmodule
```

- Testbench (program) executes in the reactive region, while design executes in the active region
- Changes in testbench (with signals triggered by clocking block) do not propagate to the active region until the clock edge arrives
- If change happens exactly at the clock edge, change available in design immediately
 - Value at 170 not seen by DUT



```
@0: req=x
@150: req=3
@250: req=1
```


Driving Signal from the Testbench: Synchronous

- We should drive interface signal in a clocking block with a synchronous drive
- Change will become available to the DUT immediately (for all the signals simultaneously)

```
program automatic test(arb_if.TEST arbif);
initial begin

    ##1 arbif.cb.request <= 3;
    ##1 arbif.cb.request <= 2;
    ##1 arbif.cb.request <= 1;
    #500 $finish;
end
endprogram : test

module arb_mod (arb_if.DUT arbif);
    initial
        $monitor("@%0t: req=%h", $time, arbif.request);
endmodule
```

```
ncsim> run
@0: req=x
@50: req=3
@150: req=2
@250: req=1
Simulation complete via $finish(1) at time 750 NS + 3
```

Program Block: Example

```
interface arb_if(input bit clk);
    logic [1:0] grant, request;
    logic reset;

    clocking cb @(posedge clk); // Declare cb
        output request;
        input grant;
    endclocking

    modport TEST (clocking cb, // Use cb
        output reset);

    modport DUT (input request, reset, clk, output grant);

endinterface

program automatic test(arb_if.TEST arbif);
initial begin
    @arbif.cb;
    arbif.cb.request <= 2'b01;
    $display("@%0t: Drove req = 01", $time);
    $display("@%0t: Grant Read 1 = %b", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: Grant Read 2 = %b", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: Grant Read 3 = %b", $time, arbif.cb.grant);
    @arbif.cb;
    $display("@%0t: Grant Read 4 = %b", $time, arbif.cb.grant);
end
endprogram : test

module arb_mod (arb_if.DUT arbif);
    always @(posedge arbif.clk)
    begin
        $display("@%0t: Inside DUT", $time);
        if (arbif.request[0])
            arbif.grant <= 2'b01;
        else if (arbif.request[1])
            arbif.grant <= 2'b10;
        else
            arbif.grant <= 2'b00;
        end
    endmodule

module top;
    bit clk;
    always #5 clk = ~clk;

    arb_if arbif(clk);

    arb_mod a1 (arbif.DUT);
    test t1(arbif.TEST);
endmodule
```

- Do the simulation and find the answer.

Program Block: Automatic vs. Static

- By default, program has tasks and functions as static

```
1 program automatic test_auto;
2   function void displayAndIncrement;
3     int auto_var;
4     auto_var++;
5     $display("auto_var: %0d", auto_var);
6   endfunction
7   initial begin
8     displayAndIncrement();
9     displayAndIncrement();
10  end
11 endprogram
```

```
ncsim> run
auto_var: 1
auto_var: 1
default_static_var: 1
default_static_var: 2
```

```
13 program test_default;
14   function void displayAndIncrement;
15     int auto_var;
16     auto_var++;
17     $display("default_static_var: %0d", auto_var);
18   endfunction
19   initial begin
20     displayAndIncrement();
21     displayAndIncrement();
22   end
23
24 endprogram
```

Coverage-driven Constrained-Random Testing (CRT)

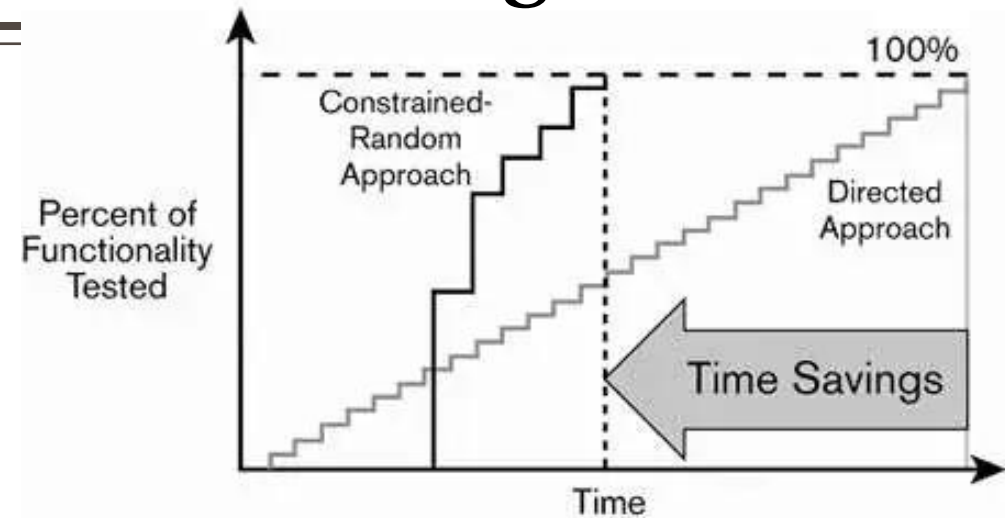
Directed vs. Constrained-Random Testing (CRT)

Directed Testing

- Testbench contains code to explicitly create scenarios
 - Provide stimulus and check (manually or with self-checks)
- Demerits:
 - High effort in creating testbench: not scalable to large designs
 - Biased to region where bug is expected

Constrained-Random Testing (CRT)

- Create environment that generates test cases automatically using pseudo-random number generator
- Restrict stimuli to valid and interesting region by using constraints



<https://www.eetimes.com/systemverilog-reference-verification-methodology-introduction/>

Advantages of CRT

- Reduces total effort and time in testing
 - Initial effort/time is high (in building the verification environment, parameterize and constrain)
 - Later hundreds of testcases can be automatically generated
- In effect: computational effort increases, manual effort decreases

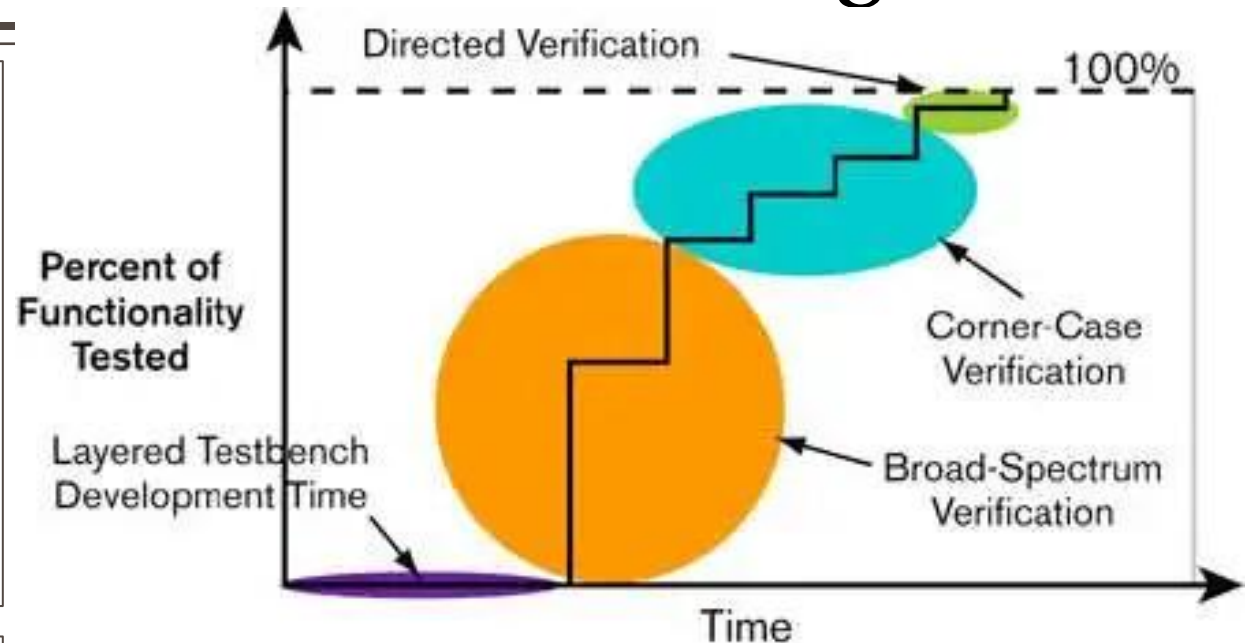
Coverage-driven Constrained-Random Testing (CRT)

How to ensure that CRT is effective?

- Use coverage-driven metrics
 - To track progress of functional verification
 - Identify verification gaps
 1. Use constraints to target uncovered regions
 2. Write directed test cases to fill the gaps

Coverage metrics

- Code-coverage
 - Crude metrics
 - Initial estimate of progress of verification
- Functional coverage:
 - Using functional coverage models
 - Preferable, but need extra effort



<https://www.eetimes.com/systemverilog-reference-verification-methodology-introduction/>

Stages in Coverage-driven CRT

1. Initial development time
2. Random Verification with some constraints to cover broad-range of function
3. Corner-case verification by refining constraints
4. Directed testing to reach 100% coverage

Randomization in SystemVerilog

Three main features

- Random Variables
- Constraint Specification
- Controlling the Random Behavior

Random Variables, Constraint, and Randomization

```
class Generator;
    rand integer marks; // declare random number
    constraint c1 { // define constraints
        marks >= 0;
        marks <= 100;
    }
    function void display;
        $display("Marks: %0d", marks);
    endfunction
endclass

program MyRand();
    initial begin
        Generator gen;
        gen = new(); // create object with random data
        for (int i=0; i <5; i++) begin
            gen.randomize(); // generate random values
            gen.display();
        end
    end
endprogram
```

```
ncsim> run
Marks: 94
Marks: 91
Marks: 28
Marks: 48
Marks: 81
```

- Variables can be declared as **rand** to enable randomization
- Constraints can be defined on randomization using **constraint**
- In-built function **randomize** gives random value to the random variables
 - If constraint cannot be satisfied, it returns 0, else 1

Coverage Definition: **coverpoint** and **covergroup**

```
class Generator;
  rand integer marks; // declare random number
  constraint c1 { // define constraints
    marks >= 0;
    marks <= 100;
  }
endclass
```

```
covergroup GradeCover;
  option per_instance = 1;
  coverpoint marks {
    bins A = {[90:100]};
    bins A_neg = {[80:89]};
    bins B = {[70:79]};
    bins B_neg = {[60:69]};
    bins C = {[50:59]};
    bins C_neg = {[40:49]};
    bins D = {[30:39]};
    bins F = {[0:29]};
  }
endgroup
```

```
function new();
  GradeCover = new();
endfunction
```

```
function void display;
  $display("Marks: %0d", marks);
endfunction
endclass
```

- Testbench samples the values of variables and expressions to measure the coverage
 - These sample locations are called **coverpoint**
- Multiple cover points that are sampled at the same time (such as when a transaction completes) are grouped into **covergroup**

- Cover group can be defined in a class, program or module
- Cover group is similar to a class and can be instantiated multiple times

- Cover group defined in a class has the member object name same as the cover group name
 - Must be created in the constructor of the class

Coverage Sampling

```
class Generator;
  rand integer marks; // declare random number
  constraint c1 { // define constraints
    marks >= 0;
    marks <= 100;
  }
  covergroup GradeCover;
    option.per_instance = 1;
    coverpoint marks {
      bins A = {[90:100]};
      bins A_neg = {[80:89]};
      bins B = {[70:79]};
      bins B_neg = {[60:69]};
      bins C = {[50:59]};
      bins C_neg = {[40:49]};
      bins D = {[30:39]};
      bins F = {[0:29]};
    }
  endgroup

  function new();
    GradeCover = new();
  endfunction

  function void display;
    $display("Marks: %0d", marks);
  endfunction
endclass
```

```
program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    gen.GradeCover.start();
    for (int i=0; i < 5; i++) begin
      gen.randomize(); // generate random values
      gen.GradeCover.sample();
    end
    gen.display();
  end
  gen.GradeCover.stop();
  $display("Coverage: %0d\\%", gen.GradeCover.get_coverage());
end
endprogram
```

Triggering a cover group

- Explicit call of the function **sample**
- Wait on some event such as **@clock** specified in cover group

Collecting coverage information

- Calling **start** and **stop** for start/stop collecting coverage information
- During simulation coverage can be obtained using:
 - **get_coverage()** gives coverage across all instances of a cover group
 - **get_inst_coverage()** gives coverage for a specific cover group instance (option.per_instance=1 must be specified)

```
ncverilog random-coverage-1.sv -coverage all -covoverwrite
```

```
Marks: 94
Marks: 91
Marks: 28
Marks: 48
Marks: 81
Coverage: 50%
```

Output of Incisive Metrics Center (IMC)

IMC (64) [Analysis - Functional: MyRand]

File View Analysis Help

Load Data Context info Source Map CoverGro... Item Advanced Local Exclude Exclude Local Exclude Smart Show Affected Show Affecting Comment Un Exclude Read Refine Save Refine Report

Context Views Analyze Locality Refinement Report

Instance (default scope): MyRand

Overall Covered Grade: 69.23% Functional Covered Grade: 50% CoverGroup Covered Grade: 50% Assertion Covered Grade: n/a

Cover Gro.. Assertions

Cover groups

Ex	UNR	Name	Overall Average Grade	Overall Covered	Enclosing Entity
		(no filter)	(no filter)	(no filter)	
		gen.GradeCover	50%	4 / 8 (50%)	MyRand

Showing 1 items

Items gen.GradeCover

Ex	UNR	Name	Overall Average Grade	Overall Covered
		(no filter)	(no filter)	(no filter)
		marks	50%	4 / 8 (50%)

Showing 1 items

Bins marks

Ex	UNR	Name	Overall Average Grade	Overall Covered	Score
		(no filter)	(no filter)	(no filter)	(no filter)
		A	100%	1 / 1 (100%)	2
		A_neg	100%	1 / 1 (100%)	1
		B	0%	0 / 1 (0%)	0
		B_neg	0%	0 / 1 (0%)	0
		C	0%	0 / 1 (0%)	0
		C_neg	100%	1 / 1 (100%)	1
		D	0%	0 / 1 (0%)	0
		F	100%	1 / 1 (100%)	1

Showing 8 items

Details marks

Col #	Name	Value
	(no filter)	(no filter)
	At Least	1
	Comment	
	Cover Group	gen.GradeCover
	CoverGroup Average Grade	50%
	CoverGroup Covered	4.0
	CoverGroup Excluded	0.0
	CoverGroup Total	8.0
	CoverGroup Total Weighted Coverage	4.0

Loaded Run: /home/sneh/system-verilog/cov_work/scope/test

Messages

Marks: 94
Marks: 91
Marks: 28
Marks: 48
Marks: 81
Coverage: 50%

Increasing number of stimuli

Marks: 94
Marks: 91
Marks: 28
Marks: 48
Marks: 81
Marks: 67
Marks: 100
Marks: 75
Marks: 51
Marks: 27
Coverage: 88%

Instance (default scope): MyRand
Overall Local Grade: 93.75% | Functional Local Grade: 87.5% | CoverGroup Local Grade: 87.5% | Assertion Local Grade: n/a

Cover groups

Name	Overall Average Grade	Overall Covered
(no filter)	(no filter)	(no filter)
gen.GradeCover	87.5%	7 / 8 (87.5%)

Bins

Name	Overall Average Grade	Overall Covered	Score
(no filter)	(no filter)	(no filter)	(no filter)
A	100%	1 / 1 (100%)	3
A_neg	100%	1 / 1 (100%)	1
B	100%	1 / 1 (100%)	1
B_neg	100%	1 / 1 (100%)	1
C	100%	1 / 1 (100%)	1
C_neg	100%	1 / 1 (100%)	1
D	0%	0 / 1 (0%)	0
F	100%	1 / 1 (100%)	2

Items

Name	Overall Average Grade	Overall Covered
(no filter)	(no filter)	(no filter)
marks	87.5%	7 / 8 (87.5%)

Details

Col #	Name	Value
	(no filter)	(no filter)
	At Least	1
	Comment	
	Cover Group	gen.GradeCover
	CoverGroup Average Grade	87.5%
	CoverGroup Covered	7.0
	CoverGroup Excluded	0.0
	CoverGroup Total	8.0
	CoverGroup Total Weighted Coverage	7.0

Loaded Run: /home/sneh/system-verilog/cov_work/scope/test

system-verilog Verilog - sneh tb_dut_race... program1 sv... program2 sv... program3 sv... random-1 sv... random-cov... IMC (64) [An...]

Adding Constraints

```

class Generator;
  rand integer marks; // declare random number
  constraint c1 { // define constraints
    marks >= 0;
    marks <= 100;
  }
  constraint c2 { // define constraints
    marks >= 30;
    marks < 40;
  }
  covergroup GradeCover;
    option.per_instance = 1;
    coverpoint marks {
      bins A = {[90:100]};
      bins A_neg = {[80:89]};
      bins B = {[70:79]};
      bins B_neg = {[60:69]};
      bins C = {[50:59]};
      bins C_neg = {[40:49]};
      bins D = {[30:39]};
      bins F = {[0:29]};
    }
  endgroup

  function new();
    GradeCover = new();
  endfunction

  function void display;
    $display("Marks: %0d", marks);
  endfunction
endclass

```

```

ncsim> run
Marks: 94
Marks: 91
Marks: 28
Marks: 48
Marks: 81
Marks: 67
Marks: 100
Marks: 75
Marks: 51
Marks: 27
Marks: 38
Coverage: 100%

```

```

program MyRand();
  initial begin
    Generator gen;
    gen = new(); // create object with random data
    gen.GradeCover.start();
    gen.c2.constraint_mode(0); // switch off constraint for corner case
    for (int i=0; i < 10; i++) begin
      gen.randomize(); // generate random values
      gen.GradeCover.sample();
      gen.display();
    end
    gen.c2.constraint_mode(1); // switch on constraint for corner case
    gen.randomize(); // generate random value within the corner case
    gen.GradeCover.sample();
    gen.display();
    gen.GradeCover.stop();
    $display("Coverage: %0d\\%", gen.GradeCover.get_coverage());
  end
endprogram

```

Bins		marks			
Abstract		Expand			
Ex	UNR	Name	Overall Average Grade	Overall Covered	Score
		(no filter)	(no filter)	(no filter)	(no filter)
		A	100%	1 / 1 (100%)	3
		A_neg	100%	1 / 1 (100%)	1
		B	100%	1 / 1 (100%)	1
		B_neg	100%	1 / 1 (100%)	1
		C	100%	1 / 1 (100%)	1
		C_neg	100%	1 / 1 (100%)	1
		D	100%	1 / 1 (100%)	1
		F	100%	1 / 1 (100%)	2

References

- Spear, Chris. SystemVerilog for Verification: A Guide to Learning the Testbench Language Features. Vol. 161. Springer, 2008.